```
cd ~/analyzelogs

nano SalesMapper.java

        package SalesCountry;

        import java.io.IOException;

        import org.apache.hadoop.io.IntWritable;

        import org.apache.hadoop.io.LongWritable;

        import org.apache.hadoop.io.Text;

        import org.apache.hadoop.mapred.*;

        public class SalesMapper extends MapReduceBase implements Mapper<LongWritable, >

        IntWritable> {

        private final static IntWritable one = new IntWritable(1);

        public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable>

        Reporter reporter) throws IOException {

        String valueString = value.toString();

        // Split by whitespace (space, tab, etc.) instead of a hyphen

        // \\s+ handles multiple spaces if they exist

        String[] SingleCountryData = valueString.trim().split("\\s+");

        // Check if there is actually data in the array before collecting

        if (SingleCountryData.length > 0) {

        // SingleCountryData[0] will now be just the IP address

        output.collect(new Text(SingleCountryData[0]), one);

        }


nano SalesCountryReducer.java

        package SalesCountry;

        import org.apache.hadoop.fs.Path;

        import org.apache.hadoop.io.*;

        import org.apache.hadoop.mapred.*;

        public class SalesCountryDriver {

        public static void main(String[] args) {

        JobClient my_client = new JobClient();

        // Create a configuration object for the job
```

```
JobConf job_conf = new JobConf(SalesCountryDriver.class);
// Set a name of the Job
job_conf.setJobName("SalePerCountry");
// Specify data type of output key and value
job_conf.setOutputKeyClass(Text.class);
job_conf.setOutputValueClass(IntWritable.class);
// Specify names of Mapper and Reducer Class
job_conf.setMapperClass(SalesCountry.SalesMapper.class);
job_conf.setReducerClass(SalesCountry.SalesCountryReducer.class);
// Specify formats of the data type of Input and output
job_conf.setInputFormat(TextInputFormat.class);
job_conf.setOutputFormat(TextOutputFormat.class);
```

nano SalesCountryDriver.java

```
package SalesCountry;
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.*;
public class SalesCountryReducer extends MapReduceBase implements Reducer<Text,>
Text, IntWritable> {
public void reduce(Text t_key, Iterator<IntWritable> values,
OutputCollector<Text,IntWritable> output, Reporter reporter) throws IOException>
Text key = t_key;
int frequencyForCountry = 0;
while (values.hasNext()) {
// replace type of value with the actual type of our value
IntWritable value = (IntWritable) values.next();
frequencyForCountry += value.get();
}
output.collect(key, new IntWritable(frequencyForCountry));
}
```

```
}
```

```
export CLASSPATH="$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-core-
2.9.0.jar:$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-common-
2.9.0.jar:$HADOOP_HOME/share/hadoop/common/hadoop-common-
2.9.0.jar:~/analyzelogs/SalesCountry/*:$HADOOP_HOME/lib/*"
```

```
~/analyzelogs$ javac -d . SalesMapper.java SalesCountryReducer.java SalesCountryDriver.java
```

```
nano manifest.txt
```

```
                    Main-Class: SalesCountry.SalesCountryDriver
```

```
jar -cfm analyzelogs.jar manifest.txt SalesCountry/*.class
```

```
start dfs.sh
```

```
start yarn.sh
```

```
~/analyzelogs$ mkdir ~/input2000
```

```
~/analyzelogs$ cp access_log_short.csv ~/input2000/
```

```
~/analyzelogs$ $HADOOP_HOME/bin/hdfs dfs -put ~/input2000 /$HADOOP_HOME/bin/hadoop jar analyzelogs.jar /input2000
/output2000
```

```
$HADOOP_HOME/bin/hdfs dfs -cat /output2000/part-00000
```